ANEXO EXAMEN DE CERTIFICACIÓN

Plan de Estudio	Desarrollo Aplicaciones FullStack Python
	Caso Rental Ski

Caso “Rental Ski (Fullstack Python)”

Debido a la reactivación de la industria turística en la región de la Araucanía es que muchos de los pequeños empresarios locales han decidido aprovechar la temporada innovando en sus respectivas áreas.

Entre estos está Carlos Araya, que ha decidido abrir un Rental de Ski en la subida del volcán Villarrica. Para esto don Carlos desea implementar un sitio web en donde sus clientes puedan reservar Ski's, y posterior a la devolución de los equipos se pueda anotar observaciones del estado de los mismos.

Es por esto que usted será responsable de elaborar una DEMO de la plataforma, de modo de convencer a don Carlos de contratar sus servicios como desarrollador(a) de software usando el stack Python/Django y el framework CSS Bootstrap. Desarrollando crear una aplicación Web escalable y robusta sin incurrir en excesivos tiempos de implementación.

Antecedentes

Luego de reunirse con Carlos, actualmente se cuentan con los siguientes antecedentes:

- El rental cuenta con un stock de equipos divididos en las siguientes categorías: Ski, Botas, Trineo, Snow Board, otros.
- Cada equipo tiene un código de identificación único, un nombre y una imagen (puede usar un campo que contenga la URL), un precio y un estado. Los estados son **disponible** y **arrendado** y **en mantención**.
- Los potenciales clientes deben ser capaces de hacer **Login** en el sistema, y **Registrarse**. Cada cliente debe tener al menos un nombre y correo electrónico.
- Los clientes acceden directamente a la página inicial (ruta: **/**). Desde acá pueden ver una lista de todos los equipos (puede usar una tabla), pudiendo filtrar por categoría. Sólo se despliegan los equipos con estado **disponible**.
- Cada cliente debe ser capaz de arrendar un equipo, y luego retornarlo. No se puede arrendar un equipo que se encuentra actualmente arrendado.
- El superadmin del sistema (usando Django-Admin) debe ser capaz de agregar nuevos equipos y categorías.
- Hay 2 tipos de usuario (además del superadmin). **Clientes** y **Operarios**.
- Los operarios deben ser capaces de poder ver todos los arriendos (ruta: **/arriendos**). Los arriendos tienen asociado un **equipo**, una **fecha**, y una **observación**. Los operarios deben poder agregar observaciones a los arriendo. Por ejemplo, si el equipo se devolvió en mal estado.
- Evidentemente no se pueden arrendar equipos en estado **arrendado** o **en mantención**.
- Al momento de arrendar un equipo (ruta: **/equipos/<id>/arrendar**) se debe especificar una fecha al menos igual a la fecha actual.

Requerimientos Funcionales del Sistema

Después de varias reuniones entre Igor y nuestro equipo técnico, los requerimientos que pudimos bajar son los siguientes:

- Desarrollar una aplicación Web basada en el prototipo propuesto en las ilustraciones
- La aplicación debe poder navegarse desde la barra superior. En la misma debe saludarse al usuario por su nombre (si está logueado).
- Las funcionalidades descritas deben estar protegidas para que sean realizadas sólo por usuarios registrados y autentificados correctamente. En el caso de que un usuario no esté registrado, el sistema debe redirigirlo a la pantalla para que haga Login o se Registre.
- La vista de administración (Django-Admin) debe tener un Banner que refleje el Rental.
- Los formularios pueden validarse en el FrontEnd o en el BackEnd. Esto queda a su discreción
- Se puede implementar la autenticación de usuarios usando Django-Auth o un sistema customizado. Pero si se debe procurar que las contraseñas estén encriptadas.
- Las rutas deben ser exactas a lo que muestran las maquetas. La excepción de esto son las rutas de autenticación. En el caso de que use otras rutas se debe especificar en el **README.txt**.

Requerimiento Final

Si un cliente operario retorna un equipo, y más tarde un operario realiza un comentario de este equipo; entonces el cliente no podrá volver a arrendar equipos durante un mes. Considere este el último requerimiento por desarrollar.

Maquetas (Mockups)

Ilustración 1: Pantalla de Registro

Ruta: /registro

Ilustración 2: Vista Home

Ruta: /

Ilustración 3: Formulario de Arriendo de Equipo

Ruta: /equipos/<id>/arrendar

Ilustración 4: Formulario de Arriendos (solo accesible para operarios)

Ruta: /arriendos

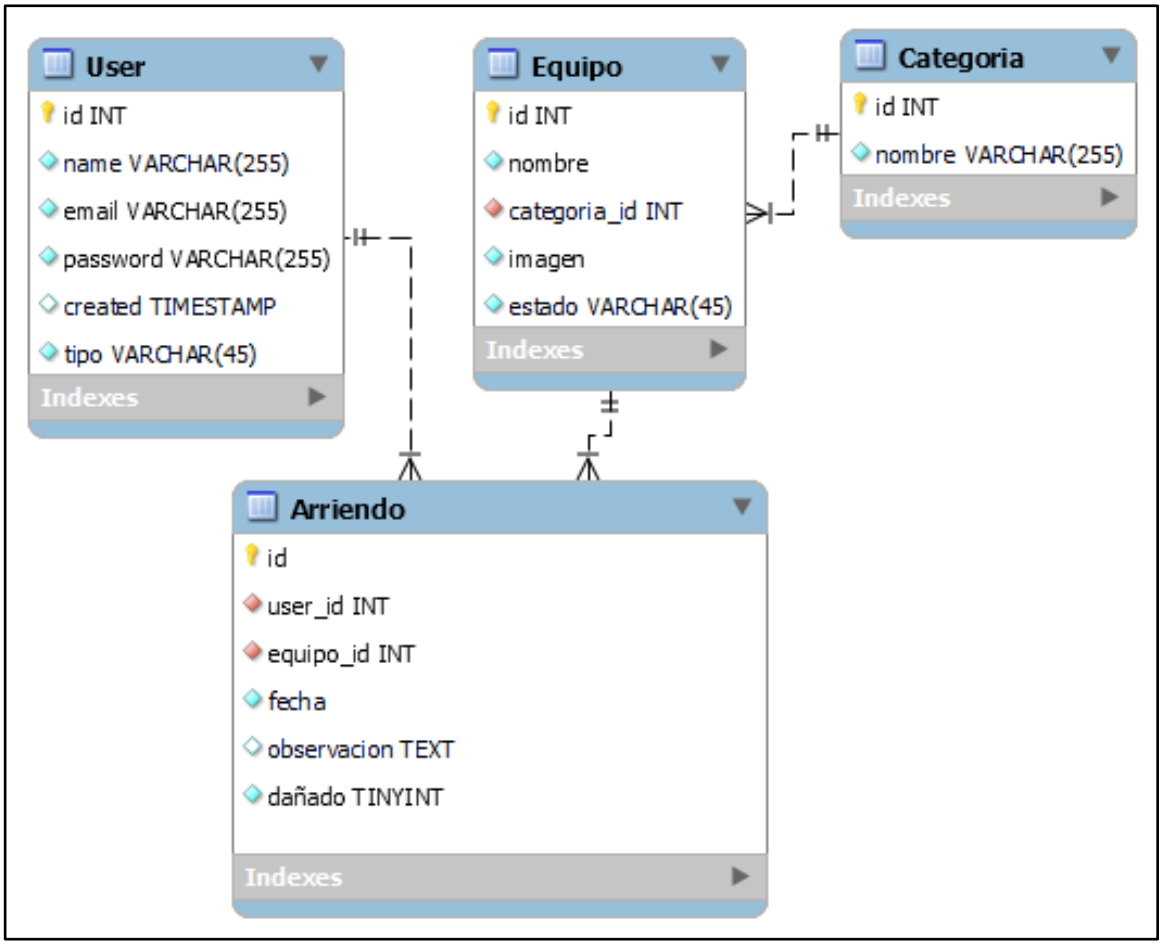
Requerimientos No Funcionales

El arquitecto del proyecto, le ha hecho las siguientes definiciones:

- El sistema debe utilizar una base de datos PostgreSql.
- El sistema debe construirse con Python/Django
- El sistema Web debe construirse bajo el patrón MVC.
- El acceso de usuarios se puede implementar con el modelo de Django Auth, o un modelo personalizado que usted prefiera.
- La capa de acceso a datos debe ser implementada con el ORM Django

Modelo de Datos

A continuación, se presenta el modelo de datos que ya ha sido diseñado por el equipo técnico del proyecto. Puede usarlo como guía para entender el problema, pero **no se preocupe** si los campos cambian de nombre al utilizar la Django ORM, o si desea quitar, agregar campos.



Entregable

Haga todos los supuestos que estime conveniente para la resolución del caso y déjelos debidamente comentados en el código fuente en caso de requerirse. Recuerde, al final del proyecto entregar lo siguiente:

- Código fuente del proyecto
- El proyecto debe llamarse exactamente **rental_ski** (sin espacios ni mayúsculas, con guión bajo).
- Export de la base de datos con esquema y datos, (le puede consultar al instructor a cargo). Este modelo debe llamarse exactamente **rental_ski.sql** (sin espacios ni mayúsculas, con guión bajo), y debe ir dentro de la carpeta del proyecto. Este export debe contener al menos 5 equipos de al menos 2 categorías distintas, Un cliente, un operario, el superadmin.

- El username del superadmin de DjangoAdmin debe ser **admin**, y su contraseña debe ser **admin**. (en caso de no ser así, debe especificar el usuario y contraseña en un README.txt en la raíz del proyecto).
- Archivo **README.txt** con todas las indicaciones para ejecutar el proyecto. Debe contener las dependencias, y los datos de los usuarios.
- Queda estrictamente prohibido compartir el código en github u otra plataforma.
- No se puede utilizar ningún framework Frontend reactivo ni Docker